



PatchPilot

every bug remembers its history

Read top to bottom. Each level goes one step deeper. Stop wherever you have enough — Level 1 is the pitch, Level 4 is the build plan.

Level 1 — The Pitch (30 seconds)

PatchPilot is a living memory system for small SaaS and engineering teams. It ingests tickets, chats, changelogs, and past fixes into Cognee, then recalls prior incidents, recommends likely root causes with evidence, and — when a release ships — detects which old workarounds have gone stale and forgets them. The result is a self-maintaining incident brain that stops teams from re-solving the same bug every week, and never recommends a fix the latest version already replaced.

`support memory` · `SaaS` · `Next.js + Python` · `Cognee remember/recall/improve/forget` · `Memory Drift`

The hook: *Most AI remembers everything. PatchPilot remembers what matters, reinforces what works, and forgets what's obsolete.*

Level 2 — What It Does (the product)

The problem

Incident knowledge is scattered across GitHub, Slack, tickets, and people's heads. Two things go wrong: teams re-investigate the same bug they already solved months ago, and AI assistants confidently return fixes that a later release has already made obsolete.

What the user does

1. **Feeds it knowledge** — past incidents and the fixes that worked.
2. **Asks about a new bug** — gets the likely root cause *plus the exact prior incidents it's based on*, so it's evidence, not a guess.
3. **Confirms the recommendation** — engineers accept or reject the suggested fix, allowing PatchPilot to reinforce successful knowledge through `improve()`.
4. **Ships a release** — PatchPilot flags the workarounds that just became wrong and forgets them.

Memory Drift — the differentiator

Every memory carries a health status. Memory Drift measures whether previously learned knowledge is still trustworthy as the software evolves.

- **Stable** — still trusted.
- **Aging** — hasn't been useful in a while, or a release touched its area.

- **Drifting** — a newer fix contradicts it; it's now wrong to recommend.

When a release comes in, PatchPilot detects drift and recommends forgetting the stale knowledge. This is what turns the project from "search with a delete button" into "a memory that maintains itself."

The 60-second demo

1. Search a bug → PatchPilot returns the **old workaround**.
2. Upload **"v2.0 release notes."**
3. Memory Drift marks the workaround **●**.
4. Click **Forget**.
5. Search the same bug again → now it returns the **new, correct fix**.

Before/after, in one minute. That switch is the moment that wins the demo.

Feature scope

- **Must build:**
 - multi-source ingest
 - `remember()` ingestion
 - `recall()` with root cause + evidence
 - Engineer feedback → `improve()`
 - Memory Graph
 - Release upload
 - Memory Drift detection
 - `forget()` obsolete workarounds
 - re-search to prove memory updated
- **Nice-to-have:** confidence scoring, memory health dashboard, incident timeline, richer graph viz, search filters, UI polish.
- **Skip:** real Slack/GitHub/Jira/Zendesk integrations, auth, multi-user, real-time sync. Use sample datasets instead.

Level 3 — How It's Built (the system)

Architecture

Next.js (App Router)	—HTTP→	FastAPI	→	Cognee (self-hosted)
the diagnosis card		lifecycle		graph + vector + sqlite
drift indicators		endpoints		.patchpilot_memory/

- **Frontend:** Next.js, a single-page dashboard. Typefaces: Space Grotesk / Inter / IBM Plex Mono. Signature element is the **diagnosis card** — root cause beside the prior incidents it was reconstructed from.
- **Backend:** FastAPI, thin lifecycle endpoints (`/remember` , `/recall` , `/feedback` , `/forget`).

- **Memory:** Cognee, self-hosted, with OpenAI `gpt-5.4-mini` for graph extraction.

The four Cognee pillars → real calls

PatchPilot action	Cognee call(s)	Notes
remember an incident	<code>add()</code> + <code>cognify()</code>	ingest text, build the graph
recall for a new bug	<code>search(GRAPH_COMPLETION)</code> + <code>search(CHUNKS)</code>	answer + evidence, fused
improve from feedback	<code>improve(feedback_alpha=...)</code>	reweights the graph
forget after a release	<code>forget(dataset=...)</code>	surgical, per-release
reset / reseed	<code>prune.prune_data()</code> + <code>prune.prune_system()</code>	clean slate for the demo

You use both the high-level V2 verbs *and* the underlying V1 pipeline (`add` / `cognify` / `search`). That depth is what scores on "Best Use of Cognee."

Why recall fuses two strategies

- `GRAPH_COMPLETION` reasons across the *connections* between incidents to produce the recommendation.
- `CHUNKS` returns the specific prior tickets that recommendation rests on.
- Together: an answer that's grounded in real evidence, not a hallucination.

Why forget can be surgical

Workarounds are stored in a **per-release dataset** (e.g. `workarounds_v1_9`), separate from durable incidents. Shipping v2.0 means a single `forget(dataset="workarounds_v1_9")` removes exactly the obsolete knowledge and leaves everything else intact.

How Memory Drift actually works (heuristics, not magic)

A release note names a component or references an incident → PatchPilot flags matching older memories:

- a newer fix **contradicts** an older workaround → ●
- a release **touches the component** an old memory is about → ● / ●
- a memory **hasn't been recalled** successfully in a while → ●
- **similar incidents now resolve differently** → ●

Drift then recommends the `forget()` (or update) through Cognee's lifecycle. Keep it explainable — the heuristic must be visible, or it reads as hand-wavy.

Level 4 — How We Ship It (the team plan)

Current status

The four-stage lifecycle (remember/recall/improve/forget) is **already scaffolded and running** in Next.js + Python + Cognee. Day 1 is not from zero — it's "add Memory Drift + file upload on top of a working base."

Work split (4 people, parallelizable)

- **Backend + Cognee:** lifecycle endpoints, dataset scoping, seed pipeline.
- **Memory Drift:** drift heuristics + release ingestion + the recommend-to-forget logic.
- **Frontend / UX:** diagnosis card, drift indicators (●●●), memory graph, polish.
- **Data + demo + PRs:** sample datasets, demo recording, and Cognee open-source PRs in parallel.

Nobody is blocked on anybody — clean four-way split.

Timeline (5 effective days + final day)

- **Day 1** — datasets ready, file upload, remember confirmed; start an approved Cognee PR.
- **Day 2** — recall + root cause + "explain why" evidence panel.
- **Day 3** — release upload, forget, memory graph, end-to-end demo working.
- **Day 4** — Memory Drift, confidence scoring.
- **Day 5** — UI polish, README, architecture diagram, demo recording, finish PRs.
- **Final day** — bug fixes, submission, blog/social posts.

Prize strategy

- **80% PatchPilot / 20% Cognee PRs**, worked in parallel.
- The PR reward track pays **\$100 per merged PR** (up to 5/person) — separate money for the whole team alongside the main build.

Cost

Only `cognify` (during ingest/seed) incurs meaningful LLM cost; recall and embeddings are negligible. Expected cost is only a few dollars for the hackathon if we keep the seed corpus small. Set a hard \$10 spending cap in the OpenAI dashboard as a safety net.

Honest risks

- **"Memory app" is the literal hackathon theme** → the field will be crowded. We don't win on the idea; we win on Drift feeling *real* and the before/after demo landing cleanly.
- **Drift heuristics can feel hand-wavy** → keep them simple and visible so judges trust them.
- **Weak demo data** — if the sample tickets, release notes, and incidents don't clearly tell the before/after story, Memory Drift won't feel convincing.
- **Deploy persistence** → Cognee writes memory to disk; Render's free filesystem is ephemeral, so attach a persistent disk or memory resets on redeploy.

The single biggest lever on our score

Make judges immediately understand that PatchPilot is impossible without Cognee's memory lifecycle. Judges should immediately understand that PatchPilot is impossible without Cognee's memory lifecycle. Memory Drift and the search → drift → forget → search-again demo should make that obvious.